

Compararea experimentală a unor algoritmi de sortare implementați în Python

Adrian-Sorin Onofrei

Facultatea de Matematică și Informatică

Universitatea de Vest din Timișoara

Informatică în limba română, grupa 4

`adrian.onofrei06@e-uvv.ro`

2026

Rezumat

Sortarea este una dintre problemele fundamentale din informatică, fiind utilizată în numeroase aplicații precum baze de date, motoare de căutare, procesarea fișierelor, analiza datelor și optimizarea altor algoritmi. Deși algoritmi de sortare au același scop, performanța lor poate varia semnificativ în funcție de dimensiunea intrării, de distribuția valorilor și de proprietățile specifice ale datelor.

Această lucrare prezintă o comparație experimentală între șase algoritmi clasici de sortare implementați în limbajul Python: Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort și Counting Sort. Algoritmii au fost testați pe mai multe tipuri de liste: liste generate aleator, liste deja sortate, liste sortate invers, liste aproape sortate și liste cu multe valori repetitive. Pentru fiecare configurație s-a măsurat timpul de execuție, folosind media a cinci rulări.

Rezultatele obținute confirmă diferențele dintre complexitățile teoretice și comportamentul practic al algoritmilor. Algoritmii cu complexitate pătratică, precum Bubble Sort, Insertion Sort și Selection Sort, devin ineficienți pentru liste mari. În schimb, Merge Sort și Quick Sort oferă performanțe mult mai bune, iar Counting Sort este foarte rapid atunci când valorile sunt întregi și aparțin unui interval restrâns.

Cuvinte cheie: algoritmi de sortare, complexitate algoritmică, Python, analiză experimentală, Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, Counting Sort.

Cuprins

1	Introducere	3
1.1	Context general	3
1.2	Motivația alegerii temei	3
1.3	Obiectivele lucrării	3
1.4	Structura lucrării	3

2	Prezentarea formală a problemei și a soluției	4
2.1	Definirea formală a sortării	4
2.2	Date de intrare și date de ieșire	4
2.3	Proprietăți urmărite	4
2.4	Algoritmii analizați	4
2.5	Complexități teoretice	5
3	Modelarea problemei și implementarea soluției	5
3.1	Limbajul de programare utilizat	5
3.2	Structura generală a programului	5
3.3	Bubble Sort	5
3.4	Insertion Sort	6
3.5	Selection Sort	6
3.6	Merge Sort	7
3.7	Quick Sort	7
3.8	Counting Sort	8
3.9	Generarea datelor de test	8
3.10	Măsurarea timpului de execuție	9
4	Studiu experimental	9
4.1	Metodologie	9
4.2	Rezultate pentru liste random	10
4.3	Rezultate comparative pentru toate tipurile de date	10
4.4	Analiza rezultatelor	10
4.4.1	Bubble Sort	10
4.4.2	Insertion Sort	10
4.4.3	Selection Sort	10
4.4.4	Merge Sort	11
4.4.5	Quick Sort	11
4.4.6	Counting Sort	11
4.5	Limitări ale experimentului	11
5	Comparația cu literatura de specialitate	11
5.1	Algoritmi simpli	11
5.2	Merge Sort	11
5.3	Quick Sort	12
5.4	Counting Sort	12
5.5	Avantaje și dezavantaje	12
6	Concluzii și direcții viitoare	12
6.1	Concluzii	12
6.2	Direcții viitoare	13
A	Cod sursă complet	15

1 Introducere

1.1 Context general

Sortarea reprezintă procesul de aranjare a elementelor unei colecții într-o anumită ordine, de obicei crescătoare sau descrescătoare. În informatică, sortarea este una dintre cele mai studiate probleme, deoarece apare ca etapă intermediară în rezolvarea multor alte probleme: căutare eficientă, eliminarea duplicatelor, clasificarea informațiilor, indexarea bazelor de date și prelucrarea datelor statistice.

Importanța sortării este dată nu doar de frecvența cu care apare în aplicații, ci și de faptul că există mai multe metode de rezolvare, fiecare cu avantaje și dezavantaje. Unele metode sunt simple și ușor de înțeles, dar lente pentru intrări mari. Altele sunt mai complexe, însă mult mai eficiente.

1.2 Motivația alegerii temei

Tema a fost aleasă deoarece algoritmi de sortare sunt un punct de pornire foarte bun pentru înțelegerea conceptelor de complexitate, eficiență și analiză experimentală. În cadrul cursurilor introductive de algoritmică se studiază adesea complexitățile teoretice, însă această lucrare urmărește să arate cum se reflectă acestea în practică.

Prin implementarea și testarea algoritmilor în Python se poate observa direct cât de mult influențează alegerea algoritmului timpul de execuție. De exemplu, pentru liste mici diferențele pot fi nesemnificative, dar pentru liste cu mii sau zeci de mii de elemente, algoritmi eficienți devin net superiori.

1.3 Obiectivele lucrării

Obiectivele principale ale lucrării sunt:

- implementarea manuală a șase algoritmi clasici de sortare;
- generarea mai multor tipuri de date de test;
- măsurarea timpilor de execuție pentru fiecare algoritm;
- compararea rezultatelor experimentale cu analiza teoretică;
- identificarea situațiilor în care un anumit algoritm este avantajos.

1.4 Structura lucrării

Lucrarea este organizată astfel: Secțiunea 2 prezintă formal problema sortării și algoritmi analizați. Secțiunea 3 descrie modelarea problemei și implementarea soluției. Secțiunea 4 conține studiul experimental și interpretarea rezultatelor. Secțiunea 5 prezintă comparația cu literatura de specialitate. Secțiunea 6 conține concluziile și direcțiile viitoare. La final sunt incluse bibliografia și anexele cu fragmente relevante de cod.

2 Prezentarea formală a problemei și a soluției

2.1 Definirea formală a sortării

Fie o listă:

$$A = [a_1, a_2, \dots, a_n]$$

formată din n numere întregi. Problema sortării constă în construirea unei permutări:

$$A' = [b_1, b_2, \dots, b_n]$$

astfel încât:

$$b_1 \leq b_2 \leq \dots \leq b_n$$

și lista A' să conțină exact aceleași elemente ca lista inițială A .

2.2 Date de intrare și date de ieșire

Date de intrare: o listă de numere întregi.

Date de ieșire: aceeași listă, ordonată crescător.

2.3 Proprietăți urmărite

Un algoritm de sortare trebuie să respecte două proprietăți esențiale:

- **corectitudine:** rezultatul final trebuie să fie ordonat crescător;
- **conservarea elementelor:** lista sortată trebuie să conțină exact aceleași valori ca lista inițială.

2.4 Algoritmii analizați

În lucrare sunt analizați următorii algoritmi:

1. Bubble Sort;
2. Insertion Sort;
3. Selection Sort;
4. Merge Sort;
5. Quick Sort;
6. Counting Sort.

Acești algoritmi au fost aleși deoarece acoperă mai multe clase importante: algoritmi simpli cu complexitate pătratică, algoritmi de tip divide et impera și un algoritm necomparativ.

2.5 Complexități teoretice

Tabela 1: Complexități teoretice ale algoritmilor analizați

Algoritm	Caz favorabil	Caz mediu	Caz nefavorabil
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$

În tabel, n reprezintă numărul de elemente, iar k reprezintă dimensiunea intervalului valorilor posibile pentru Counting Sort.

3 Modelarea problemei și implementarea soluției

3.1 Limbajul de programare utilizat

Implementarea a fost realizată în limbajul Python. Acesta a fost ales deoarece are o sintaxă clară, permite scrierea rapidă a codului și oferă module utile pentru măsurarea timpului și calculul mediei rezultatelor. În program au fost utilizate modulele `random`, `time` și `statistics`.

3.2 Structura generală a programului

Programul este alcătuit din următoarele componente:

- funcții separate pentru fiecare algoritm de sortare;
- funcții pentru generarea listelor de test;
- o funcție de măsurare a timpului de execuție;
- o funcție principală care rulează experimentele și salvează rezultatele în fișierul `rezultate.txt`.

3.3 Bubble Sort

Bubble Sort este unul dintre cei mai simpli algoritmi de sortare. Acesta parcurge lista de mai multe ori și compară elementele adiacente. Dacă două elemente vecine nu sunt în ordine corectă, ele sunt interschimbate.

Avantajul său principal este simplitatea, însă dezavantajul major este eficiența redusă. Algoritmul execută foarte multe comparații, motiv pentru care nu este potrivit pentru liste mari.

Listing 1: Implementarea Bubble Sort

```

1 def bubblesort(a):
2     a = a[:]
3     n = len(a)
4     for i in range(n):
5         for j in range(n - i - 1):
6             if a[j] > a[j + 1]:
7                 a[j], a[j + 1] = a[j + 1], a[j]
8     return a

```

3.4 Insertion Sort

Insertion Sort construiește treptat o sublistă sortată. La fiecare pas, elementul curent este inserat pe poziția corectă în partea deja sortată a listei.

Acest algoritm este eficient pentru liste mici sau aproape sortate. În cazul unei liste deja sortate, complexitatea sa devine liniară, deoarece sunt necesare foarte puține mutări.

Listing 2: Implementarea Insertion Sort

```

1 def insertionsort(a):
2     a = a[:]
3     for i in range(1, len(a)):
4         key = a[i]
5         j = i - 1
6         while j >= 0 and a[j] > key:
7             a[j + 1] = a[j]
8             j -= 1
9         a[j + 1] = key
10    return a

```

3.5 Selection Sort

Selection Sort împarte lista în două zone: o zonă sortată și una nesortată. La fiecare pas se caută cel mai mic element din partea nesortată și se mută la începutul acesteia.

Spre deosebire de Insertion Sort, Selection Sort nu profită de faptul că lista ar putea fi deja sortată. De aceea, timpul său de execuție rămâne aproximativ constant pentru tipuri diferite de intrare.

Listing 3: Implementarea Selection Sort

```

1 def selectionsort(a):
2     a = a[:]
3     n = len(a)
4     for i in range(n):
5         minidx = i
6         for j in range(i + 1, n):
7             if a[j] < a[minidx]:
8                 minidx = j
9         a[i], a[minidx] = a[minidx], a[i]
10    return a

```

3.6 Merge Sort

Merge Sort este un algoritm eficient bazat pe metoda divide et impera. Lista este împărțită recursiv în două jumătăți, fiecare jumătate este sortată, iar apoi cele două liste sortate sunt interclasate.

Algoritmul are complexitate $O(n \log n)$ în toate cazurile, ceea ce îl face stabil și predicibil. Dezavantajul său este faptul că folosește memorie suplimentară pentru interclasare.

Listing 4: Implementarea Merge Sort

```
1 def mergesort(a):
2     if len(a) <= 1:
3         return a
4
5     mij = len(a) // 2
6     st = mergesort(a[:mij])
7     dr = mergesort(a[mij:])
8
9     rez = []
10    i = j = 0
11
12    while i < len(st) and j < len(dr):
13        if st[i] < dr[j]:
14            rez.append(st[i])
15            i += 1
16        else:
17            rez.append(dr[j])
18            j += 1
19
20    rez += st[i:]
21    rez += dr[j:]
22    return rez
```

3.7 Quick Sort

Quick Sort este tot un algoritm divide et impera. Acesta alege un element pivot și împarte lista în trei părți: elemente mai mici decât pivotul, elemente egale cu pivotul și elemente mai mari decât pivotul.

În practică, Quick Sort este foarte rapid, mai ales când pivotul împarte lista relativ echilibrat. Totuși, în caz nefavorabil poate ajunge la complexitatea $O(n^2)$.

Listing 5: Implementarea Quick Sort

```
1 def quicksort(a):
2     if len(a) <= 1:
3         return a
4
5     pivot = a[len(a) // 2]
6     st = [x for x in a if x < pivot]
7     mij = [x for x in a if x == pivot]
8     dr = [x for x in a if x > pivot]
9
10    return quicksort(st) + mij + quicksort(dr)
```

3.8 Counting Sort

Counting Sort este un algoritm necomparativ, potrivit pentru liste de numere întregi aflate într-un interval relativ mic. În loc să compare elementele între ele, algoritmul numără de câte ori apare fiecare valoare.

În implementarea folosită, se determină valoarea minimă și valoarea maximă, se construiește un vector de frecvență, iar apoi se reconstruiește lista sortată.

Listing 6: Implementarea Counting Sort

```
1 def countingsort(a):
2     if not a:
3         return []
4
5     mn, mx = min(a), max(a)
6     count = [0] * (mx - mn + 1)
7
8     for x in a:
9         count[x - mn] += 1
10
11    rez = []
12    for i, c in enumerate(count):
13        rez.extend([i + mn] * c)
14
15    return rez
```

3.9 Generarea datelor de test

Pentru a compara algoritmii în condiții variate, au fost generate cinci categorii de liste:

- **random**: valori generate aleator;
- **sorted**: valori deja sortate crescător;
- **reverse**: valori sortate descrescător;
- **nearly**: liste aproape sortate, în care se fac câteva interschimbări;
- **flat**: liste cu multe valori repetitive.

Listing 7: Generarea datelor de test

```
1 def randomlist(n):
2     return [random.randint(0, n) for _ in range(n)]
3
4 def sortedlist(n):
5     return list(range(n))
6
7 def reverselist(n):
8     return list(range(n, 0, -1))
```



```

9
10 def nearlysorted(n):
11     a = list(range(n))
12     for _ in range(max(1, n // 20)):
13         i, j = random.randint(0, n - 1), random.randint(0, n - 1)
14         a[i], a[j] = a[j], a[i]
15     return a
16
17 def flatlist(n):
18     return [random.randint(0, 5) for _ in range(n)]

```

3.10 Măsurarea timpului de execuție

Pentru măsurarea performanței s-a folosit funcția `time.perf_counter()`, potrivită pentru măsurători precise ale intervalelor scurte de timp.

Listing 8: Măsurarea timpului de execuție

```

1 def measure(func, data):
2     t0 = time.perf_counter()
3     func(data)
4     return time.perf_counter() - t0

```

Fiecare test a fost rulat de cinci ori, iar valoarea raportată reprezintă media aritmetică a timpilor.

4 Studiu experimental

4.1 Metodologie

Experimentul a fost realizat pentru următoarele dimensiuni ale listelor:

10, 50, 100, 1000, 5000, 10000.

Pentru fiecare dimensiune și fiecare tip de date au fost rulați toți cei șase algoritmi. Rezultatele au fost salvate într-un fișier text, pentru a putea fi analizate ulterior.

Nu au fost folosite liste de ordinul sutelor de mii sau milioanele de elemente, deoarece algoritmi de complexitate $O(n^2)$ devin foarte lenti și ar necesita un timp de execuție prea mare.

4.2 Rezultate pentru liste random

Tabela 2: Timp de execuție pentru date random, $n = 10000$

Algoritm	Timp mediu (secunde)
Bubble Sort	3.525053
Insertion Sort	1.595017
Selection Sort	1.532276
Merge Sort	0.016754
Quick Sort	0.013402
Counting Sort	0.001970

4.3 Rezultate comparative pentru toate tipurile de date

Tabela 3: Timp de execuție pentru toate tipurile de date, $n = 10000$

Algoritm	random	sorted	reverse	nearly	flat
Bubble	3.525053	2.080148	4.513767	2.246732	3.221649
Insertion	1.595017	0.000789	3.175553	0.198003	1.283585
Selection	1.532276	1.461771	1.575399	1.467162	1.596127
Merge	0.016754	0.010204	0.010942	0.014484	0.014685
Quick	0.013402	0.008653	0.008974	0.009451	0.001559
Counting	0.001970	0.001859	0.001883	0.001921	0.000570

4.4 Analiza rezultatelor

Rezultatele experimentale evidențiază diferențe clare între algoritmi.

4.4.1 Bubble Sort

Bubble Sort este cel mai lent algoritm în majoritatea testelor. Pentru $n = 10000$, timpul de execuție depășește câteva secunde, ceea ce confirmă faptul că algoritmul nu este potrivit pentru liste mari.

4.4.2 Insertion Sort

Insertion Sort are rezultate slabe pe liste random și reverse, dar se comportă foarte bine pe liste sortate și aproape sortate. Pentru lista deja sortată de dimensiune 10000, timpul său este foarte mic, deoarece nu mai sunt necesare mutări semnificative.

4.4.3 Selection Sort

Selection Sort are un timp relativ constant indiferent de tipul de intrare. Acest lucru se explică prin faptul că algoritmul caută minimum la fiecare pas, chiar dacă lista este deja ordonată.

4.4.4 Merge Sort

Merge Sort are performanțe bune și stabile pentru toate tipurile de date. Această stabilitate este explicată de complexitatea $O(n \log n)$ în toate cazurile.

4.4.5 Quick Sort

Quick Sort este foarte rapid în testele realizate. Implementarea folosește pivotul din mijloc și separă explicit elementele egale cu pivotul, ceea ce explică performanța foarte bună pe listele cu multe valori repetitive.

4.4.6 Counting Sort

Counting Sort este cel mai rapid algoritm în majoritatea cazurilor. Totuși, rezultatul trebuie interpretat cu atenție: algoritmul este eficient deoarece datele sunt întregi și intervalul valorilor este limitat. Pentru valori foarte mari sau dispersate, consumul de memorie ar putea crește semnificativ.

4.5 Limitări ale experimentului

Experimentul are câteva limitări:

- testele au fost realizate doar în Python;
- nu s-au testat liste foarte mari;
- nu s-a măsurat consumul de memorie;
- rezultatele pot varia în funcție de calculatorul folosit;
- implementările sunt didactice, nu optimizate la nivel industrial.

5 Comparația cu literatura de specialitate

Sortarea este una dintre cele mai studiate teme din literatura informaticii. Lucrări clasice precum *Introduction to Algorithms* de Cormen, Leiserson, Rivest și Stein, respectiv *The Art of Computer Programming, Volume 3: Sorting and Searching* de Donald Knuth, prezintă în detaliu algoritmi de sortare, demonstrațiile de corectitudine și analiza complexităților.

5.1 Algoritmi simpli

Bubble Sort, Insertion Sort și Selection Sort sunt frecvent prezentați în materiale introductive deoarece sunt ușor de înțeles și de implementat. Totuși, din punct de vedere practic, aceștia au performanțe slabe pentru intrări mari, din cauza complexității $O(n^2)$.

5.2 Merge Sort

Merge Sort este asociat cu metoda divide et impera și este menționat în literatura clasică drept un algoritm eficient și stabil. Ideea de bază este împărțirea problemei în subprobleme mai mici, rezolvarea lor recursivă și combinarea rezultatelor.

5.3 Quick Sort

Quick Sort a fost introdus de C. A. R. Hoare și publicat în revista *The Computer Journal*. În literatura de specialitate este considerat unul dintre cei mai eficienți algoritmi de sortare prin comparații în practică, deși cazul nefavorabil poate avea complexitate $O(n^2)$.

5.4 Counting Sort

Counting Sort este diferit de algoritmi anteriori deoarece nu sortează prin comparații. El utilizează un vector de frecvență și poate atinge complexitate liniară $O(n+k)$ atunci când intervalul valorilor este mic. Din acest motiv, algoritmul este util în situații particulare, dar nu este o soluție generală pentru orice tip de date.

5.5 Avantaje și dezavantaje

Tabela 4: Avantaje și dezavantaje ale algoritmilor analizați

Algoritm	Avantaje	Dezavantaje
Bubble Sort	Foarte simplu, potrivit pentru înțelegerea comparațiilor și interschimbărilor	Foarte lent pentru liste mari; complexitate pătratică
Insertion Sort	Eficient pe liste mici, sortate sau aproape sortate	Ineficient pe liste mari și în caz invers sortat
Selection Sort	Implementare simplă; număr relativ redus de interschimbări	Nu profită de date deja sortate; complexitate $O(n^2)$
Merge Sort	Stabil, eficient și predictibil	Necesită memorie suplimentară
Quick Sort	Foarte rapid în practică; potrivit pentru multe situații	Poate ajunge la $O(n^2)$ în caz nefavorabil
Counting Sort	Foarte rapid pentru numere întregi într-un interval mic	Nu este general; consumul de memorie depinde de intervalul valorilor

6 Concluzii și direcții viitoare

6.1 Concluzii

În urma experimentelor realizate, se pot formula următoarele concluzii:

- algoritmi Bubble Sort, Insertion Sort și Selection Sort sunt utili pentru învățare, dar nu sunt potriviți pentru liste mari;
- Insertion Sort are performanțe foarte bune atunci când datele sunt deja sortate sau aproape sortate;
- Selection Sort are performanță relativ constantă, dar slabă;
- Merge Sort este stabil și oferă performanțe bune indiferent de tipul de date;

- Quick Sort este foarte rapid în practică, mai ales cu o alegere potrivită a pivotului;
- Counting Sort este cel mai rapid în experimentele realizate, dar depinde de natura valorilor de intrare.

Rezultatele confirmă faptul că analiza teoretică a complexității este relevantă în practică. Diferențele dintre $O(n^2)$, $O(n \log n)$ și $O(n + k)$ devin evidente pe măsură ce dimensiunea datelor crește.

6.2 Direcții viitoare

Lucrarea poate fi extinsă în mai multe direcții:

- implementarea altor algoritmi, precum Heap Sort, Radix Sort sau Shell Sort;
- compararea implementărilor Python cu implementări în C sau C++;
- măsurarea consumului de memorie;
- reprezentarea grafică a timpilor de execuție;
- testarea pe volume de date mai mari;
- compararea algoritmilor implementați manual cu funcția standard `sorted()` din Python.

Bibliografie

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, 3rd Edition, MIT Press, 2009.
- [2] Donald E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, Addison-Wesley, 1998.
- [3] Robert Sedgewick, Kevin Wayne, *Algorithms*, 4th Edition, Addison-Wesley Professional, 2011.
- [4] C. A. R. Hoare, “Quicksort”, *The Computer Journal*, Volume 5, Issue 1, pp. 10–16, Oxford University Press, 1962.
- [5] Python Software Foundation, *Python Documentation*, disponibil la <https://docs.python.org/3/>.

A Cod sursă complet

Listing 9: Codul complet al proiectului

```
1 import random
2 import time
3 import statistics
4
5 def bubblesort(a):
6     a = a[:]
7     n = len(a)
8     for i in range(n):
9         for j in range(n - i - 1):
10             if a[j] > a[j + 1]:
11                 a[j], a[j + 1] = a[j + 1], a[j]
12     return a
13
14 def insertionsort(a):
15     a = a[:]
16     for i in range(1, len(a)):
17         key = a[i]
18         j = i - 1
19         while j >= 0 and a[j] > key:
20             a[j + 1] = a[j]
21             j -= 1
22         a[j + 1] = key
23     return a
24
25 def selectionsort(a):
26     a = a[:]
27     n = len(a)
28     for i in range(n):
29         minidx = i
30         for j in range(i + 1, n):
31             if a[j] < a[minidx]:
32                 minidx = j
33         a[i], a[minidx] = a[minidx], a[i]
34     return a
35
36 def mergesort(a):
37     if len(a) <= 1:
38         return a
39     mij = len(a) // 2
40     st = mergesort(a[:mij])
41     dr = mergesort(a[mij:])
42     rez = []
43     i = j = 0
44     while i < len(st) and j < len(dr):
45         if st[i] < dr[j]:
46             rez.append(st[i])
47             i += 1
48         else:
```

```

49         rez.append(dr[j])
50         j += 1
51     rez += st[i:]
52     rez += dr[j:]
53     return rez
54
55 def quicksort(a):
56     if len(a) <= 1:
57         return a
58     pivot = a[len(a) // 2]
59     st = [x for x in a if x < pivot]
60     mij = [x for x in a if x == pivot]
61     dr = [x for x in a if x > pivot]
62     return quicksort(st) + mij + quicksort(dr)
63
64 def countingsort(a):
65     if not a:
66         return []
67     mn, mx = min(a), max(a)
68     count = [0] * (mx - mn + 1)
69     for x in a:
70         count[x - mn] += 1
71     rez = []
72     for i, c in enumerate(count):
73         rez.extend([i + mn] * c)
74     return rez
75
76 ALGORITHMS = {
77     "Bubble": bubblesort,
78     "Insertion": insertionsort,
79     "Selection": selectionsort,
80     "Merge": mergesort,
81     "Quick": quicksort,
82     "Counting": countingsort
83 }
84
85 def randomlist(n):
86     return [random.randint(0, n) for _ in range(n)]
87
88 def sortedlist(n):
89     return list(range(n))
90
91 def reverselist(n):
92     return list(range(n, 0, -1))
93
94 def nearlysorted(n):
95     a = list(range(n))
96     for _ in range(max(1, n // 20)):
97         i, j = random.randint(0, n - 1), random.randint(0, n - 1)
98         a[i], a[j] = a[j], a[i]
99     return a

```



```

100
101 def flatlist(n):
102     return [random.randint(0, 5) for _ in range(n)]
103
104 DATA_TYPES = {
105     "random": randomlist,
106     "sorted": sortedlist,
107     "reverse": reverselist,
108     "nearly": nearlysorted,
109     "flat": flatlist
110 }
111
112 def measure(func, data):
113     t0 = time.perf_counter()
114     func(data)
115     return time.perf_counter() - t0
116
117 def runtests():
118     sizes = [10, 50, 100, 1000, 5000, 10000]
119     with open("rezultate.txt", "w", encoding="utf-8") as f:
120         f.write("Observatie importanta:\n")
121         f.write("Nu au fost testate liste de ordinul zecilor sau\n")
122         f.write("sutelor de milioane de elemente.\n")
123         f.write("Motivul este limitarea hardware: memorie RAM si\n")
124         f.write("timp de executie.\n")
125         f.write("Algoritmii de complexitate O(n^2) devin\n")
126         f.write("impracticabili pentru n foarte mare.\n")
127
128         for dtype in DATA_TYPES:
129             f.write(f"\nTip date: {dtype}\n")
130             for n in sizes:
131                 data = DATA_TYPES[dtype](n)
132                 f.write(f"\n n = {n}\n")
133                 for name, func in ALGORITHMS.items():
134                     times = []
135                     for _ in range(5):
136                         times.append(measure(func, data))
137                     f.write(f"{name:10s}: {statistics.mean(times)\n")
138                     f.write(":.6f} sec\n")
139
140 if __name__ == "__main__":
141     runtests()
142     print("Rezultatele au fost salvate in rezultate.txt")

```